# TP3 – Navigation et données distantes

**React Native** · En trinôme · Projet, jalon 2

Votre projet a son architecture. Il devient maintenant une vraie application : plusieurs écrans reliés entre eux, des adresses sur le web, un détail chargé depuis votre **API publique** et des écrans qui réagissent à leur cycle de vie.

**Ce TP en bref**

- **Objectifs** : mettre en place la navigation avec Expo Router (onglets, pile, route dynamique), charger un élément par son identifiant depuis l'API avec `fetch`, gérer le cycle de vie des écrans et des listes plus riches.
- En trinôme, dans le dépôt du projet. **Lancé en séance le 29 octobre**, puis en autonomie jusqu'au 24 novembre.
- Ce jalon n'est pas noté. Ses critères font partie de la grille de production du projet, évaluée le 17 décembre.
- Le cours 3 (catalogue de séjours) vous sert de modèle.

**Qui fait quoi ?**

- **A · Catalogue** : étape 2 (charger un élément) et étape 4 (catalogue et filtres).
- **B · Navigation et compte** : étape 1 (navigation) et étape 3 (détail).
- **C · Données partagées** : la `SectionList` de l'étape 4 et l'étape 5 (cycle de vie).

# 1 La navigation

**Expo Router · Stack · Tabs**

1. Réorganisez `src/app/` :
   - une **pile racine** dans `src/app/_layout.tsx`, qui garde votre Provider ;
   - un groupe `(tabs)` avec **au moins deux onglets**, chacun avec un titre et une icône ;
   - une **route dynamique** pour le détail d'un élément, par exemple `src/app/recipes/[id].tsx` ;
   - un écran `+not-found.tsx` avec un lien de retour à l'accueil.
2. Déclarez `unstable_settings = { anchor: "(tabs)" }` dans la pile racine, pour qu'un détail ouvert par son URL garde une flèche de retour.

> Importez `Tabs` depuis `expo-router/js-tabs`. Installez les icônes avec `npx expo install @expo/vector-icons`, jamais avec `npm install` : la commande d'Expo choisit la version compatible avec votre SDK.

# 2 Charger un élément par son identifiant

**fetch · union discriminée**

1. Dans votre service, ajoutez une fonction qui charge **un seul élément** par son identifiant, par exemple `fetchRecipe(id): Promise<Recipe | undefined>`. Utilisez la route de l'API prévue pour un élément si elle existe, sinon cherchez l'élément dans la liste.
2. Comme pour la liste, vérifiez `response.ok` et convertissez la réponse en objet de votre modèle.
3. Dans `src/hooks/`, écrivez le hook qui le charge (par exemple `useRecipe(id)`). Son état est une **union discriminée** : chargement, introuvable ou succès.

**Indice : trouver la route d'un élément**

Cherchez dans la documentation de votre API un mot comme *lookup*, *details* ou *by id*. TheMealDB, par exemple, renvoie une recette avec `lookup.php?i=` suivi de son identifiant.

# 3 L'écran de détail

**useLocalSearchParams · liens profonds**

1. Depuis le catalogue, un appui sur un élément ouvre son détail avec `router.push` ou un `<Link>`.
2. Le détail lit l'identifiant avec `useLocalSearchParams` et **recharge l'élément** par un service et un hook dédiés.
3. Son titre apparaît dans l'en-tête ( `<Stack.Screen options>` ).
4. Un identifiant inconnu affiche un message clair, sans planter.

> **Le test qui compte**
>
> Sur le web, ouvrez un détail, puis **rechargez la page** (F5) et copiez son adresse dans un nouvel onglet. Le même détail doit s'afficher, avec une flèche de retour.

# 4 Le catalogue et ses filtres

**FlatList · SectionList · hooks**

1. Le catalogue affiche les données de l'API en grille sur ordinateur et en liste sur téléphone.
2. Il propose **au moins deux filtres combinables** (une recherche et une catégorie, par exemple). Leur logique est dans le modèle et dans un hook, jamais dans un composant.
3. Un des écrans utilise une `SectionList` pour présenter des éléments **groupés** (par catégorie, par date…). La fonction de regroupement est dans le modèle.
4. Chaque liste a une `keyExtractor`, un message quand elle est vide, et aucun calcul dans `renderItem`.

# 5 Le cycle de vie

**useFocusEffect · AppState**

1. Repérez un écran dont les données peuvent changer pendant qu'il est caché (un onglet de favoris, par exemple). Rechargez-le avec `useFocusEffect`.
2. Dans le README, expliquez en deux ou trois phrases **pourquoi** un `useEffect(…, [])` ne suffisait pas à cet endroit.
3. **Facultatif** : rafraîchissez une donnée au retour de l'application au premier plan, avec `AppState`.

## ✓ Avant le point d'étape

- ☐ `npx tsc --noEmit` et `npx expo lint` n'affichent aucune erreur.
- ☐ L'application se lance sur le web et sur Expo Go.
- ☐ Le README liste les écrans et leurs URL, l'API utilisée et l'explication de l'étape 5.
- ☐ Chaque membre a avancé sur sa partie et sait l'expliquer.

Poussez votre travail au fil de l'eau sur `main`. Le mercredi 25 novembre, pendant le TP4, je passe voir chaque groupe pour un **point d'étape** de 5 minutes : chacun me montre où en est sa partie. Il n'est pas noté. Il sert à repérer tôt ce qui bloque.